ARQUITECTURA · INTEGRIDAD · AUTORÍA

SPARTACO

La arquitectura de la integridad.
Veinte años para que un archivo pueda probar,
por sí mismo, que sigue siendo el mismo.

Ilver Villasmil
Investigador independiente · Miami, Florida
Veinte años de construcción · Octubre de 2026

Índice

APERTURA

Prefacio — Lo que no se construye en tres días

CAPÍTULO I

El problema que nadie resolvía

CAPÍTULO II

La Ley Ω: el suelo matemático

CAPÍTULO III

Los canales y el sello

CAPÍTULO IV

La firma y el manifiesto

CAPÍTULO V

Las dos llaves: firmar no es cifrar

CAPÍTULO VI

El usuario y la custodia de la clave

CAPÍTULO VII

El perímetro que no deja pasar nada

CAPÍTULO VIII

Killdog: la frontera que reconoce

CAPÍTULO IX

La genealogía y el arranque

CAPÍTULO X

El auditor que se audita

CAPÍTULO XI

La honestidad del método

CIERRE

Epílogo — Veinte años

APERTURA

Prefacio — Lo que no se construye en tres días

Esto no nació de un fin de semana. Nació de veinte años de preguntarme por qué un documento no puede defenderse a sí mismo. Durante dos décadas vi el mismo problema repetirse con nombres distintos: un archivo llega, alguien dice que es auténtico, y nadie —ni la persona que lo recibe ni el sistema que lo almacena— tiene forma de comprobarlo sin confiar en una palabra ajena. Vi contratos firmados en papel, escaneados, y vueltos a imprimir hasta que la firma era una mancha. Vi modelos de inteligencia artificial circular por el mundo sin que nadie pudiera decir si eran los originales o versiones manipuladas. Vi registros médicos, escrituras, patentes, obras, todos dependiendo de una cadena de custodia humana que se rompe con una simple copia.

SPARTACO es la respuesta que construí. No es una aplicación; es una arquitectura. No es una idea de tres días; es el resultado de veinte años de pulir una sola obsesión: que un archivo pueda llevar consigo la prueba de lo que es, sin depender de nadie que lo custodie.

Quiero ser claro sobre lo que tienes entre manos, porque he visto demasiadas promesas de seguridad que se deshacen al primer contacto con la realidad. SPARTACO no promete magia. Promete algo más modesto y mucho más útil: que la manipulación de un documento sea *detectable*, que la autoría sea *verificable*, y que la confidencialidad sea *real*. Tres propiedades distintas, con tres mecanismos distintos, y ninguna de ellas descansa en que alguien te prometa que todo está bien.

Lo que sigue es un recorrido por la arquitectura. No por sus pantallas, sino por sus fundamentos. Porque lo que hace que SPARTACO funcione no está en la interfaz: está en las decisiones que tomé hace años y que fui verificando, una por una, hasta que cada pieza dejó de ser una opinión y pasó a ser un hecho comprobable.

No construí un candado. Construí un sistema donde el candado, la llave y la cerradura se comprueban mutuamente.

— Ilver Villasmil

CAPÍTULO I

El problema que nadie resolvía

Existe una confusión que atraviesa toda la industria de la seguridad digital, y que durante años me pareció el corazón del asunto: la gente cree que *firmar* y *cifrar* son lo mismo. No lo son. Y creer que lo son es la razón por la que tantos sistemas parecen seguros y no lo son.

Firmar un documento responde a una pregunta: **¿quién responde por esto, y ha cambiado desde que lo firmó?** Cifrar un documento responde a otra pregunta completamente distinta: **¿quién puede leerlo?** La primera es una cuestión de autoría e integridad. La segunda es una cuestión de confidencialidad. Se pueden resolver por separado, se pueden resolver juntas, pero jamás se resuelven con el mismo mecanismo.

Durante veinte años observé cómo se mezclaban. Un sistema firmaba un archivo y creía haberlo protegido; pero el archivo seguía siendo legible para cualquiera que lo interceptara, porque firmar no oculta nada. Otro sistema cifraba un archivo y creía haber probado su origen; pero cualquiera podía cifrar cualquier cosa, así que el cifrado no decía nada sobre quién lo había hecho. La confusión no es un detalle académico: es la diferencia entre un sistema que protege y un sistema que da la ilusión de proteger.

Hay una tercera confusión, más sutil, y es la que más me costó resolver. Cuando alguien altera un archivo, ¿cómo lo sabes? La respuesta ingenua es: comparando con el original. Pero eso supone que tienes el original, que sabes dónde está, y que confías en quien te lo dio. En el mundo real, el original ya no existe: existe una copia, y otra copia, y otra, y en algún punto de esa cadena alguien cambió un número. La pregunta verdadera no es «¿es igual al original?» sino «¿puede este archivo demostrar, por sí mismo, que es el que se firmó?».

Esa reformulación lo cambia todo. Porque un archivo que puede demostrar algo por sí mismo no necesita un custodio. No necesita que confíes en nadie. Lleva su prueba encima, como un pasaporte lleva tu fotografía.

Las tres propiedades que había que garantizar

De ahí salieron las tres propiedades que SPARTACO tenía que garantizar, y que definí con precisión antes de escribir una sola línea de código.

La primera es la **integridad**: que cualquier cambio en el contenido sea detectable, sin importar cuán pequeño sea. Un bit distinto, un byte añadido, un byte eliminado. La integridad no pregunta quién cambió

el archivo; solo exige que el cambio sea imposible de ocultar.

La segunda es la **autoría**: que quien verifica pueda saber no solo que el archivo no cambió, sino quién respondió por él. Y aquí aparece la parte difícil, porque la autoría no puede basarse en una firma dibujada, ni en un sello, ni en una contraseña. Tiene que basarse en algo que sea matemáticamente imposible de falsificar sin la llave correcta. Eso se llama criptografía de clave pública, y es la única herramienta que conozco que da esa garantía.

La tercera es la **confidencialidad**: que el contenido pueda quedar ilegible para quien no tenga la llave de apertura. Y esa llave —esto es crucial— no puede ser la misma que la de autoría. Son dos poderes distintos y deben vivir en dos lugares distintos.

Un sistema que solo tiene integridad te dice que algo cambió, pero no quién lo cambió. Un sistema que solo tiene autoría te dice quién firmó, pero no protege el contenido. Un sistema que solo tiene confidencialidad te oculta el contenido, pero no te dice si es auténtico. Las tres juntas, y solo las tres juntas, forman una protección completa. Esa es la arquitectura.

Lo que había que evitar

Tan importante como definir lo que quería fue definir lo que no quería. Aprendí, a lo largo de los años, que la mayoría de los fallos de seguridad no vienen de un algoritmo roto: vienen de una decisión de diseño que parecía razonable. Así que me impuse reglas duras desde el principio.

Nada de «permitir por defecto». En SPARTACO, lo que no está explícitamente autorizado está prohibido. Un sistema que permite por defecto y luego restringe es un sistema que, ante lo desconocido, deja pasar. Un sistema que prohíbe por defecto y luego autoriza es un sistema que, ante lo desconocido, se cierra. La diferencia entre ambos es la diferencia entre una puerta abierta que a veces se cierra y una puerta cerrada que a veces se abre. En seguridad, solo la segunda es aceptable.

Nada de secretos escondidos en el algoritmo. Este es un error clásico: creer que la seguridad está en que nadie conozca cómo funciona el sistema. Es una mala apuesta, porque tarde o temprano el algoritmo se descubre, se filtra o se deduce. La seguridad tiene que vivir en la llave, no en el secreto del diseño. Esta regla me costó aceptarla, porque es humillante: significa que debes publicar cómo funciona tu sistema y confiar en que, aun sabiéndolo, nadie pueda romperlo. Pero es la única regla honesta.

Nada de confiar en la afirmación de un objeto sobre sí mismo. Un archivo no puede decir «soy auténtico» y ser creído por ello. Un componente no puede declararse a sí mismo exitoso. La autoridad tiene que venir de fuera, de un ancla que se fija antes y no se puede cambiar. Este principio, que llamé *no autoautorización*, es el que impide que un atacante simplemente escriba «válido» y pase.

Y nada de fallar abierto. Si algo va mal —si falta un archivo, si una firma es ilegible, si una excepción interrumpe el proceso— el sistema no debe interpretar el fallo como permiso. Debe interpretarlo como lo que es: un fallo. Y ante un fallo, la única respuesta segura es cerrar.

El verdadero enemigo no es el atacante brillante. Es el diseño que, ante la duda, decide confiar.

Con esas reglas escritas, y con veinte años de verlas fallar en otros sistemas y de corregirlas en el mío, empecé a construir. Lo primero que necesitaba no era código: era un suelo. Una base matemática que no dependiera de mi opinión ni de la de nadie. Y ese suelo es la Ley Ω.

CAPÍTULO II

La Ley Ω: el suelo matemático

Hay una razón por la que SPARTACO no empieza con criptografía, sino con matemática pura. Y esa razón es simple: no quería que mi sistema dependiera de mi criterio. Quería que se apoyara en verdades que fueran ciertas antes de que yo naciera y que lo sigan siendo cuando ya no esté. La aritmética es el único terreno donde eso es posible.

El corazón de la Ley Ω es una identidad que se enseña en la escuela pero que casi nadie mira de verdad: la factorización de la suma de dos cubos. Cuando tomas dos números, A y B, y sumas sus cubos, el resultado se descompone siempre —sin excepción, para todos los A y B del mundo— en el producto de dos piezas con significados muy distintos.

$$A^3 + B^3 = (A + B) \cdot (A^2 - AB + B^2) = S \cdot Q$$

La primera pieza, $S = A + B$, es lo que llamo el **canal lineal**. Es la parte que crece de forma directa, la que se ve a simple vista, la que parece contener toda la información. La segunda pieza, $Q = A^2 - AB + B^2$, es el **canal ciclotómico**, y es la parte profunda. No es un número cualquiera: es la norma de un objeto matemático llamado entero de Eisenstein, que vive en una geometría hexagonal perfecta. Es decir, dentro de una suma aparentemente trivial de cubos hay escondida la estructura del hexágono.

Esa dualidad —lo lineal y lo cíclico, lo recto y lo hexagonal— no es una curiosidad. Es la misma dualidad que aparece en todo mi sistema. Cada vez que SPARTACO verifica un archivo, calcula dos canales: uno que cubre la primera mitad del contenido y otro que cubre la segunda. No es una coincidencia estética; es la misma estructura de la factorización cúbica, aplicada a la información.

La identidad que lo ata todo

Pero hay una segunda identidad, más profunda, y es la que hace que el sistema no se pueda manipular sin romperse. Entre los dos canales existe una relación fija, que se cumple para todos los pares de números posibles:

$$4Q - S^2 = 3 \cdot (A - B)^2$$

Esta identidad es extraordinaria por una razón práctica: **liga los dos canales de forma rígida**. No puedes cambiar S sin que Q tenga que cambiar de una manera exacta y predeterminada. No son dos valores

independientes; son dos vistas del mismo hecho. Si alguien intenta presentar un S y un Q que no provienen de la misma factorización genuina, la identidad falla, y el sistema lo sabe al instante.

Junto a esta, hay otras relaciones que completan el cuadro. El producto de los dos canales es igual a la suma de los cubos: `N = S · Q`. El número tres divide al máximo común divisor de los dos canales, lo que revela que el tres —el eje ternario— está presente en toda la estructura. Y el canal profundo Q es exactamente la norma de Eisenstein de A más B omega, donde omega es la raíz cúbica compleja de la unidad.

Cuando junté todas estas identidades, me di cuenta de que no estaba mirando una fórmula: estaba mirando un **sistema de verificación**. Porque verificar no es otra cosa que comprobar que una serie de relaciones se cumplen. Y aquí tenía siete relaciones —las llamé invariantes I1 a I7— que cualquier par de números debe satisfacer si es genuino, y que ningún par manipulado puede satisfacer por accidente.

De dónde vienen los números que gobiernan todo

De esta misma estructura salen dos constantes que aparecen en todos mis módulos, y que no elegí yo: las descubrí. La primera es `α = 26/27`, y la segunda es `β = 1/27`. Su suma es exactamente uno. Y no es un uno aproximado: es la unidad exacta, la misma que exige toda teoría de probabilidad coherente.

¿Por qué 27? Porque 27 es tres al cubo, y el cubo es la figura que gobierna todo el sistema: el cubo tiene tres ejes, cada eje tiene tres posiciones, y eso da veintisiete estados. Veintiséis de esos estados forman el cuerpo principal —lo que llamo α, la parte estable, la que sostiene— y uno solo queda fuera —lo que llamo β, la singularidad, el punto de ruptura, el lugar donde la estructura se deshace—. Que α más β sea uno significa que el sistema es completo: no hay estados que se pierdan, no hay probabilidad que se escape. Todo lo que puede pasar está contado, y está contado de manera exacta.

Estas dos constantes son la semilla. En SPARTACO, antes de que cualquier módulo haga su trabajo, se comprueba que α y β cierren en la unidad. Si no cierran, el sistema no arranca. Esto puede parecer una formalidad, pero no lo es: es la garantía de que el sistema nunca opera sobre una base inconsistente. Es como verificar que los cimientos de un edificio son exactamente horizontales antes de levantar la primera pared.

El tres como eje

Hay un número que aparece una y otra vez en SPARTACO, y que quiero señalar explícitamente porque no es un detalle: el tres. Es el eje del cubo, es el exponente de la factorización cúbica, es el divisor que aparece en el máximo común divisor de los canales, es el módulo del sello neutro, y es el número de

fragmentos en que se divide el cubo para su partición fundamental. Cuando digo que SPARTACO tiene un eje, me refiero literalmente a esto: hay una dirección estructural —la ternaria— que atraviesa toda la arquitectura, desde la aritmética más abstracta hasta la forma en que un archivo se sella.

No puse el tres por gusto. Lo puso la matemática. Y esa es la diferencia entre un sistema inventado y un sistema descubierto. Yo no decidí que el eje fuera tres; la factorización cúbica me lo impuso. Mi trabajo fue reconocerlo y construir sobre él.

Con este suelo firme —la Ley Ω, las identidades, las constantes, el eje— ya tenía lo que necesitaba para construir la primera capa real del sistema: la que convierte un archivo cualquiera en algo que puede ser verificado. Esa capa son los canales y el sello.

CAPÍTULO III

Los canales y el sello

Con la matemática como suelo, el siguiente paso era convertirla en un mecanismo que funcionara sobre cualquier archivo: una fotografía, un contrato, un modelo de inteligencia artificial, un fragmento de código. Ese mecanismo son los canales y el sello. Y aquí debo detenerme en algo que aprendí de la manera difícil: *casi todo el mundo cree entender este paso y muy poca gente lo entiende de verdad*.

La idea de partida es sencilla. Si quieres saber si un archivo cambió, calculas su huella: lo pasas por una función de hash y obtienes un número. Si alguien cambia un solo bit, el número cambia por completo. Eso se llama SHA-256, es un estándar mundial, y es la base de la integridad digital. Hasta aquí, nada nuevo.

Pero fíjate en la trampa, porque es el punto que casi nadie ve: **el número es público y cualquiera puede calcularlo**. Si alguien altera tu archivo, puede calcular su nuevo número y presentártelo. Y si tu sistema solo compara números, tú aceptas el archivo alterado sin saberlo. El hash detecta cambios solo si tienes el número original guardado en algún lugar de confianza. Es decir, el hash, por sí solo, no te libera del custodio. Te ata a él.

Aquí es donde SPARTACO hace algo distinto, y donde la Ley Ω deja de ser teoría y se vuelve herramienta. En lugar de calcular una sola huella sobre el archivo entero, lo divido en dos mitades y calculo una huella sobre cada una. A esas dos huellas las llamo canales, S y Q, siguiendo la nomenclatura de la factorización cúbica. La huella completa la llamo núcleo. Y la elección no es arbitraria: es la misma estructura de dos canales que la matemática me había impuesto, ahora aplicada a la información.

El sello: la marca que el archivo lleva encima

Pero los canales todavía tienen el mismo problema que un hash normal: son números que hay que guardar en algún sitio. Y aquí viene la parte que considero más elegante del sistema, y que tardé años en dar con ella. En lugar de guardar el número aparte, SPARTACO **hace que el archivo se modifique a sí mismo hasta que su propia huella cumpla una condición**.

El procedimiento es el siguiente. Al archivo se le añade, al final, una marca de texto. Esa marca contiene un número —el contador de intentos— y se prueba con uno, con dos, con tres, hasta que el hash del archivo con esa marca cumple una propiedad concreta: que sea divisible por tres. Cuando se encuentra ese

número, el archivo queda sellado. El archivo sellado tiene, por construcción, una huella que es múltiplo del eje ternario.

Esta operación tiene un nombre en criptografía: es una prueba de trabajo, del mismo tipo que usan las cadenas de bloques para minar. Pero aquí no sirve para gastar energía ni para competir: sirve para que el archivo **lleve su propio sello**. La marca no es un secreto. Cualquiera puede leerla. Pero no puede falsificarla sin recalcularla, y recalcularla exige que el archivo esté completo y exacto.

Y hay una segunda marca, aún más fina, que llamo el sello neutro. No basta con que el archivo sea divisible por tres: además, SPARTACO calcula ocho valores a partir del archivo, dividiéndolo en ocho fragmentos y midiendo, en cada uno, cuántos bits ceros iniciales tiene su hash. Esos ocho valores forman una firma estructural del contenido que se guarda dentro del manifiesto. No es un número que se copia de otro archivo: es un número que se **deriva** de este archivo concreto y de ningún otro.

Cuando entendí que estos ocho valores eran una huella única por archivo, supe que tenía la pieza que me faltaba. Porque significa que el sello no se puede trasplantar: si tomas el sello de un archivo y se lo pegas a otro, no cuadra. El sello pertenece a su contenido como la forma de una llave pertenece a su cerradura.

Lo que el sello no es

Aquí debo ser riguroso, porque he visto cómo este punto se malinterpreta —incluso yo mismo lo malinterpreté al principio, y lo corregí—. El sello **no es un secreto y no es, por sí solo, una defensa**. Cualquiera que conozca el mecanismo puede recalcular el sello de cualquier archivo, igual que yo lo recalculé cuando estudié el sistema. Eso es inevitable, porque el verificador necesita recalcularlo para comprobar. El sello no guarda nada; comprueba algo.

Su función es **detectar cambio**, y para eso es perfecto: si un solo bit del contenido se altera, los ocho valores cambian y el sello deja de cuadrar. Pero no impide que alguien fabrique un archivo nuevo con su sello nuevo. Para eso hace falta la firma, y de eso trata el capítulo siguiente.

Lo que quiero que quede claro es la división del trabajo. El sello es el **número**. La firma es la **promesa**. El número dice «este contenido es este contenido y no otro». La promesa dice «y yo, con mi llave, respondo por él». Ninguno de los dos sustituye al otro, y creer que el sello basta es el error que cometen los sistemas que parecen seguros y no lo son.

El sello ata el contenido a sí mismo. La firma ata el contenido a una persona. Son

dos nudos distintos, y hacen falta los dos.

Con los canales calculados, el archivo sellado y la firma estructural derivada, SPARTACO ya tenía lo necesario para construir el objeto que viaja con el archivo y que lo hace verificable: el manifiesto. Y ese manifiesto, más que un archivo, es una declaración firmada.

CAPÍTULO IV

La firma y el manifiesto

Cuando proteges un archivo con SPARTACO, el sistema no te devuelve el archivo modificado y ya está. Te devuelve dos cosas: el archivo sellado y un **manifiesto**. Y el manifiesto es, en el fondo, una declaración jurada en formato de texto: un documento que dice exactamente qué es el archivo, y que lleva al final una firma que nadie más puede imitar.

El manifiesto tiene dos partes. La primera se llama el cuerpo, y contiene toda la información del archivo: su núcleo, sus dos canales, su tamaño exacto, los ocho valores del sello neutro, el identificador del artefacto, la fecha de emisión, el algoritmo usado y el identificador de la clave pública que firma. La segunda parte se llama la firma, y es una cadena de ciento veintiocho caracteres que, por sí sola, vale más que todo el cuerpo junto: es la prueba de que quien emitió el manifiesto poseía la llave privada correspondiente.

La relación entre las dos partes es lo que hace que el sistema funcione. La firma cubre **todo** el cuerpo. No una parte, no un resumen: todo. Si alguien cambia un solo carácter del cuerpo —el núcleo, una fecha, un identificador—, la firma deja de ser válida. Es un sello que se rompe si tocas cualquiera de las palabras que sella.

Firmar de verdad, no de palabra

Aquí debo explicar por qué la firma de SPARTACO no es una firma cualquiera, porque es el punto donde muchos sistemas fallan. Hay tres maneras de «firmar» un documento digital, y solo una es real.

La primera es la firma dibujada: una imagen de una rúbrica pegada al pie del documento. Eso no prueba nada. Cualquiera puede copiar la imagen. La segunda es la firma con contraseña: el sistema guarda una clave secreta y comprueba que quien presenta el documento la conoce. Eso es mejor, pero tiene un defecto fatal: la misma clave sirve para firmar y para verificar, así que cualquiera que pueda verificar puede, en principio, firmar. La tercera es la firma criptográfica de clave pública, y es la única que da la garantía completa.

En la firma de clave pública existen dos llaves relacionadas matemáticamente. Una es privada y la guarda solo el dueño. La otra es pública y se puede repartir a todo el mundo. Lo extraordinario es la asimetría: con la privada se firma, y con la pública se verifica, pero **es computacionalmente imposible deducir la privada a partir de la pública**. Esto no es una promesa de marketing: es una consecuencia

de las matemáticas de las curvas elípticas, y resiste el ataque de todas las computadoras del mundo juntas durante más tiempo que la edad del universo.

SPARTACO usa el estándar Ed25519, que es uno de los esquemas de firma de clave pública más sólidos y más rápidos que existen hoy. Cada usuario tiene su par de llaves. Cuando protege un archivo, el sistema construye el cuerpo del manifiesto, lo serializa de una forma canónica —es decir, de una forma única y reproducible, sin ambigüedad de formato— y lo firma con la llave privada. El resultado es una firma que solo ese usuario, con esa llave, pudo haber producido.

```
firma = Ed25519(llave_privada).firmar( cuerpo_canónico )
```

Y la verificación hace el camino inverso. Toma el cuerpo del manifiesto, lo vuelve a serializar de la misma forma canónica, y comprueba con la llave pública si la firma corresponde. Si corresponde, sabe dos cosas a la vez: que el cuerpo no cambió desde que se firmó (integridad) y que quien lo firmó poseía la llave privada (autoría). Una sola operación, dos garantías.

La serialización canónica, un detalle que lo decide todo

Hay un detalle técnico que quiero contar porque es donde muchos sistemas se rompen sin que nadie lo note, y donde SPARTACO puso especial cuidado: la **serialización canónica**.

Un mismo objeto puede escribirse de muchas maneras distintas. Los campos pueden ir en un orden o en otro; puede haber espacios o no haberlos; los caracteres especiales pueden codificarse de una forma o de otra. Si firmas el objeto escrito de una manera y luego lo verificas escrito de otra, la firma falla aunque el contenido sea el mismo. Es una fuente inagotable de errores sutiles.

Por eso SPARTACO impone una única forma de escribir: los campos ordenados alfabéticamente, sin espacios innecesarios, con la codificación fijada. Así, el mismo cuerpo siempre produce exactamente los mismos bytes, y la firma siempre cuadra. Además, antes de serializar, el sistema valida cada campo: que los tipos sean los correctos, que los números estén dentro de rango, que no haya valores extraños. Rechaza lo que no es canónico en lugar de aceptarlo silenciosamente. Es la diferencia entre un sistema que falla ruidosamente y uno que falla en silencio; y en seguridad, el silencio es lo peor que puede pasar.

Lo que el manifiesto demuestra y lo que no

Quiero ser honesto sobre los límites, porque una arquitectura que se vende como infalible es una arquitectura que miente. Un manifiesto firmado demuestra tres cosas: que el archivo coincide con la declaración, que la declaración la emitió el dueño de la llave, y que nada de eso cambió desde la firma. Lo que

no demuestra es que el contenido sea verdadero. Si yo firmo un documento que dice que el cielo es verde, la firma prueba que yo lo dije, no que sea cierto. La criptografía garantiza la autoría de una afirmación, no su veracidad. Confundir ambas cosas es otro de los errores clásicos, y quiero dejarlo fuera de mi sistema desde el principio.

Y hay una segunda honestidad que me impongo: un manifiesto firmado protege tanto como proteja la llave. Si la llave privada se filtra, cualquiera puede emitir manifiestos válidos, y el sistema entero cae. Esto no es un defecto de SPARTACO: es la ley de hierro de toda la criptografía de clave pública. La seguridad de un sistema de firma es exactamente igual a la seguridad de su llave privada, ni más ni menos. Por eso dedico tanto esfuerzo a la custodia de esa llave, y por eso el capítulo siguiente trata de las dos llaves distintas que SPARTACO usa, y por qué nunca deben confundirse.

La firma no dice que el documento sea verdad. Dice quién responde por él. Y esa distinción es la que separa la seguridad de la superstición.

CAPÍTULO V

Las dos llaves: firmar no es cifrar

Llego al capítulo que considero el más importante para entender SPARTACO de verdad, y el que más me costó hacer entender a lo largo de los años. Es la distinción entre las dos llaves. Y no es una distinción técnica menor: es la diferencia entre un sistema que protege la autoría y un sistema que protege el contenido. Confundirlas es el error que arrastra a la mayoría de los sistemas que he estudiado.

Volvamos a la pregunta básica. Cuando proteges un archivo, ¿qué quieres exactamente? La respuesta honesta es que quieres dos cosas distintas, y a menudo no te das cuenta de que son distintas. Quieres, por un lado, poder demostrar que el archivo no cambió y quién lo emitió. Y quieres, por otro, que el archivo no pueda ser leído por quien no deba. Lo primero es autoría e integridad. Lo segundo es confidencialidad. Y son dos problemas con dos soluciones.

La llave que firma no abre nada

Este es el punto que quiero que quede grabado. La llave privada que SPARTACO usa para firmar —la llave Ed25519— **no sirve para descifrar el contenido**. No es que sea mala para eso: es que no tiene ninguna relación con el cifrado. Son dos mundos matemáticos separados. Tener la llave de firma te da el poder de declarar autoría; no te da el poder de leer.

Esta separación es deliberada, y es una de las decisiones de las que más orgulloso estoy. Significa que puedes demostrar que un archivo es auténtico sin poder leerlo, y puedes leer un archivo sin poder demostrar quién lo emitió. Los dos poderes viven en dos lugares distintos, y quien tenga uno no tiene el otro por defecto.

Para el cifrado, SPARTACO usa un camino completamente distinto. En lugar de una llave pública y una privada, usa una **clave maestra de entrada** —lo que llamo el IKM, treinta y dos bytes de material secreto— y a partir de ella deriva varias claves especializadas. Es decir: de una sola semilla secreta, el sistema genera varias llaves, cada una con un oficio distinto.

La herramienta que hace esta derivación se llama HKDF, y es un estándar. Lo que hace es tomar la semilla y una etiqueta, y producir una llave determinista: la misma semilla con la misma etiqueta siempre produce la misma llave, y semillas distintas producen llaves distintas. Y aquí está la elegancia: SPARTACO usa etiquetas distintas para cada dominio, de modo que de una sola semilla nacen cuatro llaves separadas.

```
IKM → K_auth (autenticación) IKM → K_def (defensa interna) IKM → K_doc
(cifrado del documento) IKM → K_rec (recuperación)
```

Estas cuatro llaves son matemáticamente independientes. Conocer una no revela ninguna de las otras. Y esto es crucial: significa que un atacante que descubra la llave de autenticación no puede descifrar el documento, y uno que descubra la llave del documento no puede falsificar la autenticación. La separación de dominios convierte una sola semilla en cuatro compartimentos estancos.

El cifrado que ata la llave al contenido

Para cifrar el contenido, SPARTACO usa AES-256-GCM, que es uno de los cifrados más respetados del mundo. Pero lo interesante no es el algoritmo —ese ya existía— sino cómo lo ata al archivo. La llave de cifrado no se usa directamente: se liga al contenido. En concreto, la llave del documento se deriva de manera que depende del núcleo del archivo. Esto significa que **otro documento implica otra llave**. No puedes tomar la llave de un archivo y usarla en otro, porque la llave misma depende de qué archivo es.

Además, el cifrado usa un valor llamado nonce —un número que solo se puede usar una vez con una llave dada— que se deriva también del contexto: del núcleo, de la sal, del identificador del artefacto y del identificador de la autoridad. El resultado es que recifrar el mismo contenido en el mismo contexto es idempotente, mientras que cifrar contenido distinto nunca reutiliza un nonce. Esto evita una de las vulnerabilidades más sutiles de los cifrados de flujo, donde reutilizar un nonce puede revelar el contenido.

Todo esto está ligado por una regla que el sistema verifica explícitamente: que las cuatro llaves sean distintas entre sí. Si por algún error de implementación dos dominios produjeran la misma llave, el sistema lo detectaría y se negaría a operar. Es la clase de comprobación que parece paranoica hasta que salva un sistema.

Por qué esta separación importa tanto

Quiero cerrar este capítulo con la razón práctica, porque no es un ejercicio académico. Imagina que no tuvieras esta separación, y que la misma llave sirviera para firmar y para cifrar. Entonces, cualquiera que necesitara verificar una firma —es decir, cualquiera— tendría en sus manos la capacidad de descifrar el contenido. La verificación, que debería ser un acto público, se convertiría en un acto que revela secretos. Y al revés: cualquiera que pudiera leer el contenido podría falsificar la autoría.

Al separar las llaves, SPARTACO consigue que verificar sea un acto seguro que cualquiera puede hacer, sin que ello comprometa la confidencialidad. Puedes entregar tu llave pública a todo el mundo, y con ella cualquiera puede comprobar que un archivo es auténtico, pero nadie puede leer su contenido ni falsificar tu firma. Esa es la propiedad que hace que el sistema sea realmente utilizable en el mundo real, donde necesitas que terceros verifiquen sin que ello te exponga.

Una llave para responder por el archivo. Otra para abrirlo. Nunca la misma, porque el que verifica no debe poder leer, y el que lee no debe poder falsificar.

Con las dos llaves separadas —la de firma y la de cifrado— y con los cuatro dominios derivados del IKM, ya tenía la arquitectura criptográfica completa. Pero faltaba la parte que en la práctica lo decide todo: dónde vive cada llave, quién la custodia, y cómo se protege de la persona más peligrosa de todas, que a menudo es el propio dueño distraído. De eso trata el capítulo del usuario.

CAPÍTULO VI

El usuario y la custodia de la clave

Dije antes que la seguridad de un sistema de firma es exactamente igual a la seguridad de su llave privada. Eso convierte la custodia de esa llave en el problema central. No el algoritmo, no el cifrado, no la interfaz: la llave. Si la llave se filtra, todo lo demás es decoración. Así que dediqué una capa entera del sistema a resolver una sola pregunta: ¿dónde vive la llave privada, y cómo se protege de todo, incluido su dueño?

SPARTACO es un sistema multiusuario, y eso lo cambia todo. No hay una sola llave: cada persona tiene su par. Cuando alguien se crea una cuenta, el sistema genera su par de llaves y guarda **solo la pública**. De la pública, además, calcula una huella: un resumen de su contenido que sirve como identificador. Ese identificador —que en el sistema aparece como el campo de clave pública del manifiesto— es lo que permite saber, sin revelar nada, cuál es la llave que firmó.

Quiero detenerme aquí porque este punto me confundió a mí mismo durante años, y estoy seguro de que confunde a cualquiera que se acerque al sistema por primera vez. El identificador de clave pública que aparece en el manifiesto no es la llave pública: es su huella. Es como el número de documento en un pasaporte: identifica al pasaporte sin ser el pasaporte. Así, un verificador puede comprobar que la firma corresponde a la llave correcta comparando huellas, sin necesidad de exponer la llave completa. Es una práctica estándar en seguridad, y es la correcta.

La llave privada no se guarda en el disco

Y aquí está la decisión de diseño que considero más valiente de toda la arquitectura, y la que más veces tuve que defender. **La llave privada no se persiste.** No se guarda en un archivo permanente. No se escribe en la base de datos. Vive únicamente en memoria, durante la sesión en la que el usuario la carga, y desaparece cuando esa sesión termina.

Esto tiene un coste: cada vez que el usuario quiere operar, tiene que aportar su llave. Pero la ganancia es enorme. Significa que si alguien roba el disco duro, no encuentra la llave privada, porque nunca estuvo allí. Si alguien hace una copia de seguridad del sistema, la llave no viaja en esa copia. La llave solo existe mientras su dueño la usa, y solo en la memoria volátil de su máquina. Es la diferencia entre dejar las llaves de tu casa debajo del felpudo y llevarlas siempre encima.

Lo mismo ocurre con el material de cifrado. El IKM —la semilla maestra de la que se derivan las cuatro claves— tampoco se guarda como tal. El sistema guarda solo **referencias**: identificadores que apuntan a

la clave pero no la contienen. Es como guardar el número de una caja de seguridad en lugar de la combinación. La combinación —el IKM— se aporta cuando hace falta y se olvida después.

Lo que el sistema sabe y lo que no

Quiero ser explícito sobre el modelo de confianza, porque es donde muchos sistemas fallan por vaguedad. En SPARTACO, la capa del usuario no cifra, no firma, no deriva claves y no decide por encima de las demás. Es una fachada: relaciona a la persona con sus llaves, guarda sus perfiles, sus conjuntos de llaves y sus artefactos, y compone las operaciones de las demás capas. No implementa criptografía propia. Reutiliza la que ya existe, sin duplicarla ni modificarla.

Esa separación de responsabilidades no es un lujo arquitectónico: es una defensa. Significa que un error en la capa del usuario no puede romper la criptografía, porque esa capa no toca la criptografía. Y significa que la criptografía se puede auditar por separado, sin arrastrar consigo la lógica de cuentas y perfiles. Cada capa hace una cosa, y la hace completa.

El sistema guarda, para cada usuario, un perfil con su llave pública y su huella, sus conjuntos de llaves —que son referencias, no secretos—, sus artefactos y el estado de su vinculación. Y comprueba, en cada operación, que la huella almacenada corresponda realmente a la llave pública: que el resumen coincida con el contenido. Si alguien manipulara el archivo de perfil, la comprobación lo detectaría. La huella es, en el fondo, la integridad del propio perfil.

La llave privada no se guarda: se lleva. Y lo que no está en el disco, no se puede robar del disco.

Con las llaves en su sitio y el usuario en el suyo, quedaba construir el guardián: la capa que decide, operación por operación, si algo está permitido o no. Esa capa es el perímetro, y es donde la filosofía de «prohibido por defecto» se vuelve concreta.

CAPÍTULO VII

El perímetro que no deja pasar nada

Toda arquitectura de seguridad necesita un guardián en la puerta. Alguien que, ante cada operación, decida si se permite o se niega. Y aquí hay una decisión que parece técnica pero es filosófica: ¿el guardián permite por defecto y niega lo malo, o niega por defecto y permite lo bueno?

SPARTACO eligió lo segundo, y con una convicción que he defendido durante años. El guardián del sistema —lo que llamo el perímetro— niega por defecto. Si la operación no está en su lista de operaciones conocidas, la niega. Si falta una comprobación, la niega. Si algo lanza una excepción en mitad del proceso, la niega. No hay un caso, ni uno solo, en que el silencio o la duda se interpreten como permiso.

Esto contrasta con la mayoría de los sistemas, que hacen lo contrario. El patrón habitual es permitir y luego restringir: se parte de un estado abierto y se añaden reglas para cerrar cosas. Es más fácil de construir y más cómodo de usar, pero tiene un defecto estructural: cada cosa que se te olvide restringir queda abierta. En cambio, si partes de un estado cerrado y añades permisos explícitos, cada cosa que se te olvide autorizar queda cerrada. Ante el olvido —y siempre hay olvidos—, uno falla hacia la seguridad y el otro falla hacia la exposición.

La condición de paso

El perímetro no es una puerta con una sola cerradura. Es una puerta con varias, y todas tienen que abrirse a la vez. La condición para que una operación pase se puede escribir con precisión, y lo hice así porque una condición que no se puede escribir es una condición que no se puede auditar.

$$\texttt{ALLOW} \iff T \land D \land O \land R \land C \land Q$$

Cada término es una comprobación independiente. El primero, la validez estructural: que los datos tengan la forma y los tipos correctos. El segundo, la autorización previa: que la operación haya sido permitida en su fase de entrada. El tercero, la ligadura de operación: que la operación que sale sea la misma que entró, sin sustituciones. El cuarto y el quinto, las ligaduras de recurso y de contexto: que el recurso y el contexto no hayan cambiado entre la entrada y la salida. Y el sexto, la postcondición: que el resultado final sea exactamente verdadero, no un valor que se le parezca.

Todos estos términos se combinan con una conjunción: si uno solo falla, todo falla. No hay compensaciones, no hay excepciones, no hay «casi». Es la lógica más dura que existe, y es la correcta

para un guardián.

La capa Ω dentro del perímetro

Y aquí es donde la matemática de la que hablé en los primeros capítulos vuelve a aparecer, ahora como herramienta de seguridad. Dentro del perímetro hay una comprobación que no se basa en firmas ni en hashes, sino en los invariantes cúbicos de la Ley Ω. Cuando una operación lleva consigo un par de valores —los llamo A y B—, el sistema calcula a partir de ellos los dos canales de la factorización cúbica y comprueba que se cumplan las siete identidades.

Es decir: la aritmética de la suma de cubos, esa que parece un ejercicio de escuela, funciona aquí como una barrera. Si los valores presentados no provienen de una factorización genuina, alguna de las siete identidades falla y la operación se niega. Y como estas identidades se cumplen para todo par posible, la barrera no tiene agujeros: no hay ningún par que pase sin ser coherente.

Quiero ser preciso sobre lo que esto aporta y lo que no, porque no quiero vender humo. Esta capa no añade secreto: cualquiera que conozca la matemática puede calcular los canales correctos, igual que yo los calculé al estudiar el sistema. Lo que aporta es **coherencia estructural**: garantiza que los valores que atraviesan el perímetro formen un conjunto que se sostiene por sí mismo, que no haya piezas pegadas de sitios distintos. Es una comprobación de consistencia interna, y en un sistema complejo la consistencia interna es un valor real, no un adorno.

Lo que sí es una barrera dura, en cambio, es el resto del perímetro: la firma, la integridad de los canales de hash, la ligadura entre entrada y salida, y sobre todo el hecho de que ninguna excepción se interpreta como permiso. Ahí es donde el sistema no cede. Si el proceso se rompe en cualquier punto, el resultado no es «permiso por accidente»: es negación limpia, con un código de motivo que dice exactamente qué falló.

Un guardián que ante la duda deja pasar no es un guardián. Es una puerta giratoria.

El perímetro decide sobre operaciones. Pero había un problema más profundo que quería resolver: no ya si una operación está permitida, sino si un objeto *pertenece* al sistema o es ajeno a él. Y para eso construí una pieza distinta, inspirada en algo que la biología resolvió hace mil millones de años: el reconocimiento de lo propio y lo extraño. Esa pieza se llama Killdog.

CAPÍTULO VIII

Killdog: la frontera que reconoce

Hay una pregunta que ningún sistema de firmas responde bien, y que sin embargo es fundamental: ¿este objeto pertenece a mi sistema, o es un extraño? No me refiero a si está bien firmado, sino a si *forma parte* de lo que el sistema considera propio. Un objeto puede estar perfectamente firmado y, aun así, ser ajeno: pertenecer a otro sistema, a otra identidad, a otro conjunto. Distinguir lo propio de lo extraño es lo que hace el sistema inmune de un cuerpo vivo, y es lo que quise que hiciera una pieza de SPARTACO.

La llamé Killdog, y es probablemente la parte más conceptual de toda la arquitectura. No es un antivirus, no es un cortafuegos, no es un escáner de amenazas. Es un **sistema de reconocimiento de frontera**: recibe un objeto, lo observa, y decide si pertenece a su identidad o no. Y lo hace sin consultar a nadie, sin pedir permiso, sin depender de un centro de control.

La idea de fondo es la de un sistema inmune distribuido. En un cuerpo vivo, no hay un órgano central que decida qué es una amenaza: hay células repartidas por todo el cuerpo, cada una capaz de reconocer lo propio y atacar lo extraño. Eso mismo quise para SPARTACO: no un guardián único, sino una capacidad que puede existir en muchos lugares a la vez, cada instancia con su propia presencia pero todas compartiendo la misma ley.

La ley de reconocimiento

El reconocimiento no es una corazonada: es una comprobación formal. Killdog evalúa el objeto contra cinco criterios, y solo si los cinco se cumplen lo declara propio.

El primero es la **identidad**: que el objeto sea quien dice ser, comprobando su identificador y su huella de contenido. El segundo es el **contrato**: que tenga la forma correcta y que su autenticidad corresponda a la del ancla. El tercero es la **pertenencia**: que el objeto figure entre los miembros que el sistema reconoce. El cuarto es la **coherencia**: que su estado, sus errores, sus capacidades y sus dependencias sean consistentes con lo declarado. Y el quinto es la **señal**: que lleve la marca que lo identifica.

Si los cinco se cumplen, el objeto es propio. Si falla uno, es ajeno. Y aquí hay una regla que considero esencial y que quiero destacar: **el contrato no autoriza una contradicción con la coherencia**. Es decir, un objeto puede tener la forma perfecta y la firma correcta, y aun así ser rechazado si su contenido es incoherente. Esto evita el error clásico de creer que «tengo la firma válida, luego puedo pasar». La firma prueba la forma; la coherencia prueba el fondo; y el fondo manda.

Lo que hace que Killdog sea distinto

Hay tres decisiones en Killdog de las que estoy especialmente satisfecho, porque resuelven problemas que he visto mal resueltos en otros sistemas.

La primera es la **inmutabilidad del ancla**. El ancla es el patrón contra el que Killdog compara: contiene lo que el sistema considera propio. Y ese ancla se congela al construir la instancia. No se puede modificar después. El propio objeto lo comprueba: cada vez que alguien consulta el ancla, el sistema recalcula su huella y, si detecta que cambió, lanza un error. Es decir, el ancla se defiende a sí misma contra la manipulación.

La segunda es la **no autoautorización**. Un objeto no puede declararse a sí mismo válido. Si un objeto llega diciendo «yo me autorizo», Killdog lo rechaza de inmediato, sin siquiera evaluar el resto. Esto puede parecer obvio, pero es una de las vulnerabilidades más comunes: sistemas que aceptan la palabra de un objeto sobre sí mismo. En SPARTACO, la evidencia describe, pero no decide; la autoridad viene del ancla, nunca del objeto.

La tercera es la **no gobernabilidad**. Killdog no obedece a nadie. No se puede desactivar, no se puede reconfigurar, no responde a órdenes externas. Esto suena radical, pero es exactamente lo que se quiere de un sistema inmune: que no pueda ser convencido de bajar la guardia. Un guardián que puede ser sobornado no es un guardián.

La multiplicación que no multiplica el poder

Y aquí está la parte más elegante, que quiero explicar con cuidado porque es una idea que tardé en formalizar. Killdog puede existir en muchos lugares a la vez. Puedes tener muchas instancias, repartidas por distintos puntos del sistema, todas funcionando en paralelo. Es lo que llamo multiplicación de cuerpos.

Pero —y esto es lo importante— **multiplicar los cuerpos no multiplica la autoridad**. Todas las instancias comparten exactamente la misma ley, el mismo ancla, la misma capacidad. Lo único que cambia entre ellas es su posición operativa, su lugar. Es como tener muchas copias del mismo juez: no tienes muchos jueces, tienes un juez presente en muchos lugares. La ley es una; las presencias son muchas.

Esto resuelve un problema real de los sistemas distribuidos, donde a menudo se confunde la replicación con la autoridad: se cree que tener más nodos da más poder de decisión, y eso lleva a inconsistencias. En Killdog, la distinción es nítida: la capacidad es inmutable y única; el cuerpo es múltiple y móvil. Una

instancia puede cambiar de lugar —puede trasladarse— pero no puede cambiar lo que es. Puede cambiar dónde está; no puede cambiar qué es.

Muchos cuerpos, una sola ley. La presencia se multiplica; la autoridad, nunca.

Con el perímetro decidiendo sobre operaciones y Killdog reconociendo lo propio, faltaba una pieza que uniera todo a lo largo del tiempo: un registro que permitiera seguir la historia de cada artefacto, desde su creación hasta su verificación. Esa pieza es la genealogía, y junto con ella el sistema de arranque seguro, que es donde SPARTACO se vuelve más ambicioso.

CAPÍTULO IX

La genealogía y el arranque

Un archivo no existe en el vacío: nace, se transforma, se copia, se verifica. Y a lo largo de esa vida, quiero poder reconstruir su historia. No solo si es auténtico ahora, sino de dónde viene, qué se hizo con él, en qué orden. Esa es la idea de la genealogía: darle a cada artefacto un árbol que registre su ascendencia.

La genealogía se construye con una estructura que llamo ZSQ, y que es un árbol de partición ternaria. Cada nodo del árbol representa una porción del contenido, y cada nodo se divide en tres hijos: Z, S y Q. El identificador de cada nodo no es un número arbitrario: se calcula como el hash del identificador del artefacto combinado con su ruta en el árbol. Así, cada nodo tiene una identidad determinista y verificable: dado el artefacto y la posición, cualquiera puede calcular qué identificador le corresponde.

```
node_id = SHA-256( artifact_id ‖ path )
```

El árbol se detiene cuando las porciones son suficientemente pequeñas, y en las hojas se guardan los tres valores del canal correspondiente. Lo que resulta es una estructura que no solo verifica el contenido, sino que puede demostrar que una porción concreta pertenece a un artefacto concreto, sin necesidad de revelar el resto. Es la misma idea que usan las estructuras de datos merkle, pero con la partición ternaria que impone la Ley Ω: tres canales, no dos, porque el eje del sistema es el tres.

Proteger el arranque, no solo los archivos

Pero donde la genealogía se vuelve verdaderamente ambiciosa es en el arranque. Porque proteger un documento está bien, pero proteger el propio sistema que arranca es otra cosa. Si alguien puede modificar el kernel antes de que se cargue, de nada sirve que los archivos estén firmados: el sistema que los verifica ya está comprometido.

SPARTACO resuelve esto con una idea que me parece la más profunda de toda la arquitectura: **no modifica el artefacto que se va a cargar**. El kernel en el disco sigue siendo un kernel normal, ejecutable, sin cambios. Lo que se firma no es el kernel, sino un *descriptor* que contiene su huella: su núcleo, sus canales, su tamaño. Verificar consiste en leer el archivo vivo, recalcular su huella, y comprobar que coincide con la del descriptor firmado. Si un solo byte del kernel cambió, la huella no cuadra y el sistema no arranca.

Esta decisión —firmar la huella y no el archivo— tiene una consecuencia que valoro especialmente: la fuente de verdad es el archivo real que el firmware va a cargar, no una copia sellada que podría desincronizarse. En muchos sistemas, la verificación se hace contra una copia, y si el archivo real cambia sin que la copia lo sepa, el sistema arranca con algo que no fue verificado. Aquí no: lo que se verifica es exactamente lo que se ejecuta.

Y hay un último paso que cierra el círculo. SPARTACO no vive «al lado» del sistema: se **incrusta dentro del artefacto**. Es decir, el kernel, la capa de seguridad y las constantes se empaquetan juntos en una sola imagen, y la huella que se firma es la de esa imagen final. Así, no hay forma de sustituir una parte sin romper la huella del todo. Es la diferencia entre proteger las piezas y proteger el objeto completo.

Antes de que cualquier operación de arranque ocurra, el sistema comprueba que las constantes cierren: que α y β sumen exactamente uno. Si no cierran, no hay arranque. Es la misma disciplina que en el resto del sistema, llevada al nivel más bajo: verificar los cimientos antes de levantar nada.

No se firma el archivo y se confía en una copia. Se firma la huella y se verifica el archivo real. Lo que se ejecuta es lo que se comprobó.

Con la genealogía y el arranque, SPARTACO ya cubría desde el nivel más bajo hasta el más alto. Pero quedaba una pregunta incómoda, y era la que más me obsesionaba: ¿cómo sé que todas estas garantías que afirmo son verdad? No basta con escribirlas; hay que demostrarlas. Y para demostrarlas construí un auditor que se audita a sí mismo, y al que dediqué el capítulo siguiente.

CAPÍTULO X

El auditor que se audita

Cualquiera puede escribir que su sistema es seguro. La diferencia entre una afirmación y un hecho es la demostración. Y aquí tomé una decisión que, me doy cuenta ahora, es lo que separa a SPARTACO de la mayoría de los proyectos de seguridad que he visto: construí un auditor que no se cree nada, ni siquiera a sí mismo.

El principio que gobierna este auditor es brutalmente simple, y lo escribí como regla dura: **no hay veredicto positivo sin una falsación ejecutada**. Es decir, el sistema no puede declarar que una propiedad se cumple simplemente porque no encontró un fallo. Tiene que haber intentado activamente romperla, y haber fracasado. La ausencia de prueba no es prueba de ausencia, y este auditor lo sabe.

Esto contrasta con la manera habitual de hacer pruebas, donde se comprueba que las cosas funcionan. Aquí se hace lo contrario: se intenta que fallen. Se mutan los datos, se alteran los bytes, se sustituyen las firmas, se falsifican los manifiestos, se prueban tipos hostiles, se lanzan ataques de concurrencia, se manipulan los estados. Y solo si el sistema resiste todos esos intentos —ejecutados de verdad, no imaginados— se concede el veredicto positivo.

El auditor se calibra a sí mismo

Pero aquí está la parte que considero más rigurosa, y la que me costó más implementar. Un auditor que no se audita es un auditor en el que no se puede confiar. Si mi herramienta de verificación tuviera un error, diría «todo bien» aunque no lo estuviera, y yo lo creería. Así que el auditor se somete a sí mismo a una calibración.

Antes de juzgar al sistema, el auditor se prueba contra casos de control cuyo resultado conoce de antemano. Se le da un caso que debe fallar, y se comprueba que efectivamente falla. Se le da un caso que debe pasar, y se comprueba que pasa. Se le somete a una mutación conocida, y se verifica que la detecta. Si el auditor no pasa su propia calibración, queda descalificado y su veredicto no vale. Es como un juez que, antes de juzgar, tiene que demostrar que distingue el bien del mal en casos de prueba.

Además, el auditor lleva una contabilidad estricta. Cada caso que se genera debe estar contabilizado: o se ejecutó, o se explica por qué no. Cada capacidad descubierta debe recibir su campaña. Cada propiedad debe recibir su veredicto. Y la ecuación contable tiene que cerrar: el número de casos generados debe ser exactamente igual a los ejecutados más los no ejecutados explicados. Si sobra o falta uno solo,

el auditor se descalifica a sí mismo. Es la misma disciplina del resto del sistema —que nada se pierda, que todo cuadre— aplicada a la propia auditoría.

El veredicto honesto: no cualificado no es un fracaso

Y aquí hay una distinción que quiero subrayar, porque es la marca de un sistema maduro. El auditor tiene un veredicto llamado «no cualificado». No es lo mismo que «fallo» ni que «éxito»: significa que la prueba no se pudo ejecutar de forma concluyente. Si una capacidad no tiene una condición falsable, o si una campaña no se pudo completar, el veredicto no es «pasa»: es «no cualificado». El sistema prefiere admitir que no sabe antes que afirmar que sabe.

Esto es extraordinariamente raro. La mayoría de los sistemas de prueba colapsan todo en «pasa» o «falla», y en el proceso, lo que no se pudo probar se cuenta como éxito por defecto. Aquí no: lo no probado se declara no probado, y eso se propaga al veredicto final, que puede decir con precisión qué propiedades se demostraron, cuáles fallaron y cuáles quedaron sin cualificar. Es la diferencia entre un informe que dice «todo bien» y un informe que dice exactamente qué sabe y qué no.

Sobre esta base se ejecutan más de mil doscientos casos de prueba, y todos pasan. Pero el número no es lo importante: lo importante es que ese número sale de un proceso que se niega a aprobar nada sin haber intentado romperlo, y que se descalifica a sí mismo si su contabilidad no cuadra. Es una demostración, no una promesa.

Un sistema que no intenta romperse a sí mismo no está probado. Está intacto por falta de intentos.

Con el auditor demostrando las garantías, la arquitectura quedaba completa. Y sin embargo, al presentarla, hay una última cosa que debo hacer, y es la más difícil de todas: ser honesto sobre sus límites. Porque la diferencia entre un sistema que inspira confianza y uno que la exige es, precisamente, que el primero no teme decir lo que no puede hacer. De eso trata el último capítulo.

CAPÍTULO XI

La honestidad del método

He descrito una arquitectura que considero sólida, y lo he hecho con convicción porque llevo veinte años construyéndola. Pero terminaría traicionando el propósito de todo esto si no dijera también lo que el sistema no hace, y lo que la propia arquitectura, sometida a escrutinio, reveló sobre sí misma. La honestidad no es una debilidad en seguridad: es la única base sobre la que se puede construir confianza.

Empecemos por lo más importante, porque es la ley que gobierna todo lo demás. La seguridad de SPARTACO es exactamente igual a la seguridad de la llave privada de cada usuario. No hay margen, no hay red de seguridad, no hay segunda oportunidad. Si la llave privada se filtra, quien la tenga puede emitir manifiestos válidos para cualquier contenido, y el sistema los aceptará como auténticos. Esto no es un defecto: es la naturaleza de toda la criptografía de clave pública, y negarlo sería mentir. Lo que SPARTACO puede hacer —y hace— es proteger esa llave con todos los medios a su alcance: no persistiéndola, no escribiéndola en disco, manteniéndola solo en memoria durante la sesión. Pero no puede hacerla indestructible, porque nada lo es.

La segunda honestidad es sobre el sello. Cuando estudié mi propio sistema con ojo crítico, descubrí algo que conviene dejar claro: el sello neutro, esos ocho valores derivados del contenido, es **recomputable por cualquiera**. No es un secreto. Yo mismo lo recalculé, y cualquiera con el mismo conocimiento puede hacerlo. Eso es inevitable, porque el verificador necesita recalcularlo. Su función no es guardar un secreto, sino detectar cambios. Y para eso es impecable. Pero no debe confundirse con una defensa criptográfica, porque no lo es. La defensa criptográfica es la firma. El sello es el testigo, no el guardián.

Lo que el escrutinio corrigió

Y aquí debo contar algo que me parece esencial para entender cómo se construye un sistema de verdad, porque no todo fueron aciertos. Cuando sometí mi propia arquitectura a un análisis adversarial externo —algo que todo sistema debería recibir y casi ninguno recibe—, aparecieron malentendidos que se convirtieron en hallazgos, y hallazgos que se convirtieron en correcciones.

Uno de esos malentendidos fue el formato del identificador de clave pública. Durante un tiempo se creyó que el campo del manifiesto estaba mal etiquetado, que declaraba una llave distinta de la que realmente firmaba. Fue un error de lectura, y la corrección fue esclarecedora: el identificador no es la llave pública, es su huella. Es decir, el hash de la llave. Así, el manifiesto declara la huella correcta de la llave que firma, y quien verifica compara huellas. Es la práctica estándar, y estaba bien desde el principio. El

hallazgo desapareció cuando se entendió el formato, y ese proceso —hallazgo, escrutinio, corrección, claridad— es exactamente cómo debe funcionar la seguridad.

Otro punto que el escrutinio aclaró es la separación de las capas. Yo había construido varios mecanismos que a primera vista parecían redundantes: los canales de hash en una capa, los invariantes cúbicos en otra, el reconocimiento de frontera en una tercera. El análisis reveló que no eran redundantes: eran **capas distintas que responden a preguntas distintas**. Una comprueba que el archivo coincide con su declaración; otra decide si una operación está permitida; otra determina si un objeto pertenece al sistema. Tres preguntas, tres mecanismos. Lo que parecía duplicación era, en realidad, especialización.

Lo que el sistema no puede hacer

Hay tres cosas que SPARTACO no puede hacer, y quiero decirlas sin ambigüedad, porque prometerlas sería deshonesto.

No puede demostrar que el contenido de un documento sea verdadero. Puede demostrar que alguien concreto lo firmó y que no ha cambiado desde entonces, pero la verdad del contenido es un asunto aparte, que ninguna matemática puede resolver. Un manifiesto firmado prueba la autoría de una afirmación, no su veracidad.

No puede proteger la llave privada de su dueño. Puede evitar que se guarde, puede mantenerla solo en memoria, puede no escribirla nunca en el disco. Pero si el dueño la copia a un lugar inseguro, o la comparte, o su equipo está comprometido, el sistema no puede impedirlo. La custodia de la llave es, en última instancia, una responsabilidad humana, y ninguna arquitectura puede sustituir esa responsabilidad.

Y no puede ser más fuerte que las decisiones de diseño que lo componen. Cada elección —usar Ed25519, derivar cuatro dominios, no persistir la llave, negar por defecto— es un eslabón. La cadena es tan fuerte como el más débil, y aunque he procurado que todos los eslabones sean firmes, la humildad exige reconocer que un sistema se juzga por sus peores decisiones, no por sus mejores.

Un sistema honesto no dice «es imposible romperme». Dice «esto es lo que garantizo, esto es lo que no, y aquí está la prueba de lo primero».

Con estas tres limitaciones declaradas —y no escondidas— la arquitectura queda completa y honesta. No es un sistema que promete el cielo. Es un sistema que hace exactamente lo que dice, y que dice exactamente lo que hace. Y eso, después de veinte años, es lo que más valoro de él.

CIERRE

Epílogo — Veinte años

Veinte años. Cuando empecé, no tenía una arquitectura: tenía una intuición. La intuición de que un documento debería poder defenderse a sí mismo, sin depender de la palabra de nadie. No sabía cómo. No sabía si era posible. Solo sabía que el problema era real y que nadie lo estaba resolviendo de verdad.

Los primeros años fueron de búsqueda. Aprendí criptografía, aprendí teoría de números, aprendí a leer sistemas de seguridad y a encontrar sus grietas. Descubrí que la mayoría de las soluciones eran parciales: resolvían la integridad pero no la autoría, o la autoría pero no la confidencialidad, o la confidencialidad pero se apoyaban en un custodio central que anulaba todo lo demás. Ninguna me satisfacía, porque ninguna resolvía el problema completo.

Después vinieron los años de construcción, que fueron años de descartar. Escribí y borré más de lo que conservé. Cada pieza que hoy existe fue probada, cuestionada, y solo sobrevivió si resistía el escrutinio. La Ley Ω no nació como una herramienta de seguridad: nació como una investigación sobre la estructura de los cubos, y solo después, al ver las identidades que se cumplían siempre, entendí que tenía en las manos algo que podía servir para verificar la coherencia de un sistema. Ese momento —darme cuenta de que la matemática que había descubierto podía ser el suelo de una arquitectura de integridad — fue el punto de inflexión.

Y luego vinieron los años de endurecimiento, que son los más duros y los menos visibles. No se trata de añadir funciones: se trata de quitar debilidades. Cada línea se revisó buscando cómo romperla. Cada mecanismo se sometió a ataques. Cada afirmación se convirtió en una prueba ejecutable. Aprendí que un sistema de seguridad no se construye añadiendo capas, sino eliminando confianzas. Cada confianza que eliminas es una puerta que cierras.

Hoy, cuando miro lo que hay, veo una arquitectura que hace tres cosas y las hace completas. Protege la integridad: cualquier cambio es detectable. Protege la autoría: quién respondió por el archivo es verificable por cualquiera, sin comprometer nada. Y protege la confidencialidad: el contenido queda ilegible para quien no tenga la llave de apertura, que es distinta de la de firma. Tres propiedades, tres mecanismos, y todos construidos sobre el mismo suelo matemático.

No voy a decir que es perfecto. No lo es, y ningún sistema lo es. Tiene límites que he declarado, y tiene un punto de fallo —la llave privada— que ninguna arquitectura puede eliminar, porque es la naturaleza misma de la criptografía de clave pública. Lo que puedo decir es que es honesto: hace lo que dice,

dice lo que hace, y demuestra sus afirmaciones con pruebas que se niegan a aprobar nada sin haber intentado romperlo.

Lo que tienes en las manos no es un producto que salió de un fin de semana. Es el resultado de veinte años de no conformarme con soluciones parciales, de no aceptar confiar en la palabra de nadie, de insistir en que un archivo pueda llevar consigo la prueba de lo que es. Es un sistema construido con la convicción de que la seguridad no es una característica que se añade al final, sino una propiedad que se teje en cada decisión desde el principio.

Y si algo he aprendido en estos veinte años, es que la verdadera seguridad no está en el candado más fuerte. Está en no necesitar confiar en nadie para saber que el candado sigue cerrado. Eso es lo que SPARTACO hace posible. No porque sea mágico, sino porque cada pieza, cada canal, cada firma, cada invariante, se comprueba a sí misma y a las demás. La confianza no se pide: se demuestra.

Eso es lo que construí. Y eso es lo que te entrego.

— Ilver Villasmil
Miami, Florida · Octubre de 2026

La confianza no se pide. Se demuestra.

SPARTACO · Arquitectura de la integridad

Veinte años de construcción · Ilver Villasmil · 2026